

HF01 - Helyes zárójelezés ellenőrzése

Kiíró: Németh Dániel István

Adatszerkezetek és Algoritmusok 2023 őszi

Beadási határidő: 2023 Szeptember 21, 23:59

Feladat

A feladat az első előadáson és gyakorlaton tanultak felhasználásával egy olyan program megírása, amely ellenőrzi, hogy a bemenetként kapott `string`-ben található kifejezés helyes-e zárójelezési szempontból, azaz:

- Minden nyitó és csukó zárójelnek megvan a párja (pl.: `()` nem helyes)
- Mindhárom zárójel típust kezelni kell `{ }`, `( )`, `[ ]`, az ezeken kívüli karaktereket nem kell figyelembe venni
- Nincsen átfedés az különböző típusú zárójelek által határolt blokkok között (pl.: `[()]` nem helyes)

Az alábbi szignatúrájú függvényt kell megvalósítani a kiadott skeleton kód `bracketchecker.cpp` fájljában:

```
bool BracketChecker::checkCorrectness(const std::string& expression) {  
    //TODO solve homework by deadline  
}
```

Feltételezhetjük, hogy a függvény bemenete (`const std::string& expression`) legfeljebb 1024 karakterből áll.

A megoldás működésének ellenőrzését néhány teszt is segíti, melyek a `main.cpp` fájlban találhatóak. Ezen tesztek eredményei a console-ra íródnak ki a program futtatásakor, a console-ra kiírt pontszám a sikeresen lefutott tesztek számát jelzi, nem a feladatra kapott pontszámot. **FONTOS: A tesztek csak segítik a helyes működés ellenőrzését, de nem fednek le minden esetet, így akár helyesen lefutó tesztek esetén is lehet hiba az algoritmusban, megoldásban. Ajánlott írni saját teszteket a működés további ellenőrzésére.**

A pontozásnál a kódminőség, és az algoritmus hatékonysága is számít a kód helyes működése mellett. Alkalmazzuk a kurzus során tanult módszereket és adatszerkezeteket! A kódba magyarázó kommentek is kerüljenek, amelyek leírják a kód működését! A megoldáshoz STL-t tilos használni. A megadott flagekkel (`-Werror -Wall -Wextra -pedantic`) nem forduló kód esetén a feladat nem értékelhető (0p).